

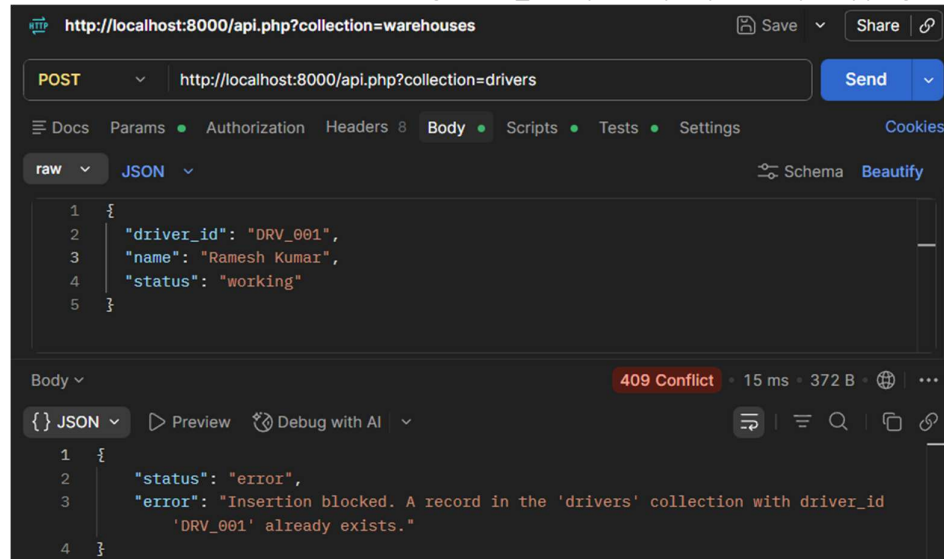
TESTING:

Test 1: The Duplicate ID Blocker (Primary Key Protection)

Defensive API Architecture: Duplicate Primary Key Prevention

Action: the seeder just created DRV_001, we will attempt to create it again.

Here, In drivers collection we are using driver_id as primary key field(By mapping to object iDs)



```
http://localhost:8000/api.php?collection=warehouses
POST http://localhost:8000/api.php?collection=drivers
Send

raw JSON
1 {
2   "driver_id": "DRV_001",
3   "name": "Ramesh Kumar",
4   "status": "working"
5 }
```

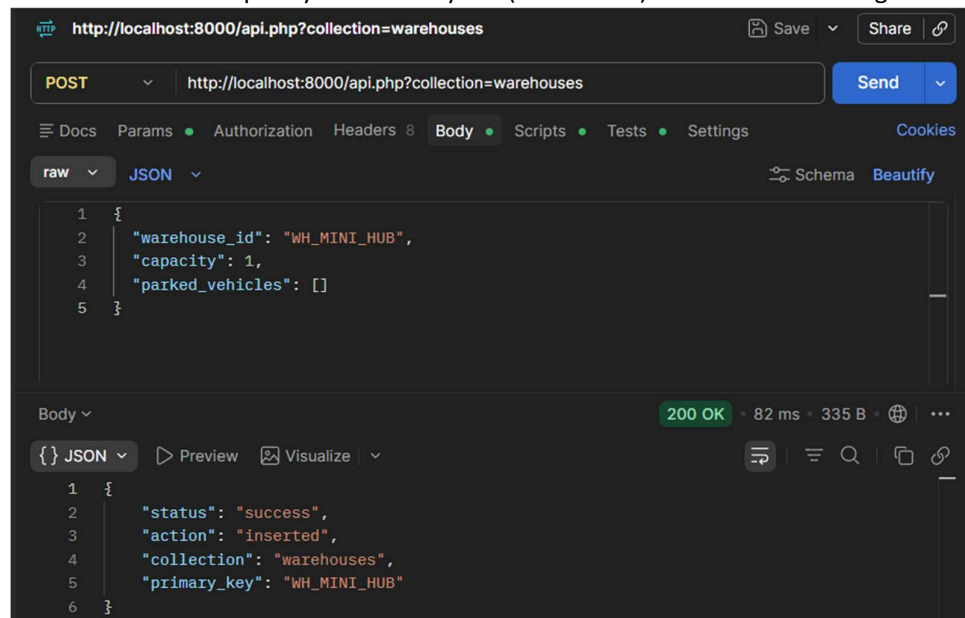
```
Body
409 Conflict 15 ms 372 B
1 {
2   "status": "error",
3   "error": "Insertion blocked. A record in the 'drivers' collection with driver_id 'DRV_001' already exists."
4 }
```

This test verifies the API's duplicate prevention mechanism. By mapping the custom driver_id directly to MongoDB's immutable _id primary key field, the database natively intercepts duplicate entries. As shown, the application successfully blocks the insertion and returns an HTTP 409 Conflict error, protecting the system from identical records.

Test 2: Real-Time Array Capacity Validation

Dynamic One-to-Many Relationships: Capacity Overflow Protection

The Action: We will quickly create a tiny hub(warehouse) and overflow it using the seeded trucks.

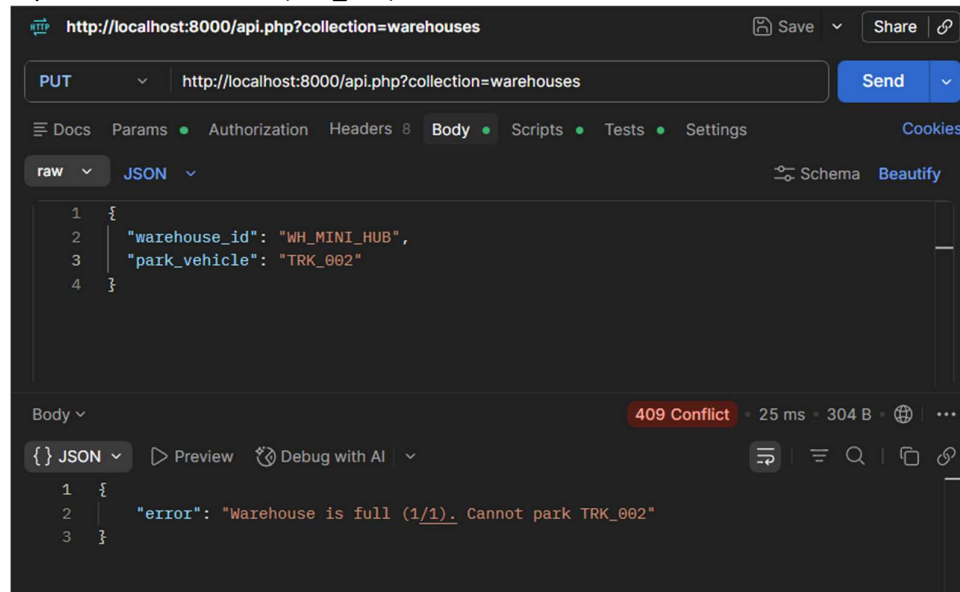


```
http://localhost:8000/api.php?collection=warehouses
POST http://localhost:8000/api.php?collection=warehouses
Send

raw JSON
1 {
2   "warehouse_id": "WH_MINI_HUB",
3   "capacity": 1,
4   "parked_vehicles": []
5 }
```

```
Body
200 OK 82 ms 335 B
1 {
2   "status": "success",
3   "action": "inserted",
4   "collection": "warehouses",
5   "primary_key": "WH_MINI_HUB"
6 }
```

Try to add another truck(TRK_002)



This test verifies real-time capacity validation within the database. By dynamically checking the length of the parked_vehicles array against the warehouse's capacity limit before executing an \$addToSet command, the API successfully blocks overflow attempts. The resulting HTTP 409 Conflict response ensures the NoSQL architecture accurately reflects physical constraints.

Test 3: Multi-Entity Dispatch & Idempotency Lock

3. State Management and Multi-Entity Junction Logging

The Action: Open your terminal and run the master dispatch command linking four of your seeded entities together.

The Success Run: (CLI)

```
PS C:\Users\K MANWITH REDDY\OneDrive\Documents\Desktop\MongoDB Task\fleet-engine> php cli.php dispatch DRV_002 TRK_003 SHP_004 WH_005
SUCCESS: Driver DRV_002 is moving Shipment SHP_004 to WH_005 via Truck TRK_003!
PS C:\Users\K MANWITH REDDY\OneDrive\Documents\Desktop\MongoDB Task\fleet-engine>
```

The Idempotency Run: Hit the **Up Arrow** and run that exact same command a second time immediately.

```
PS C:\Users\K MANWITH REDDY\OneDrive\Documents\Desktop\MongoDB Task\fleet-engine> php cli.php dispatch DRV_002 TRK_003 SHP_004 WH_005
SUCCESS: Driver DRV_002 is moving Shipment SHP_004 to WH_005 via Truck TRK_003!
PS C:\Users\K MANWITH REDDY\OneDrive\Documents\Desktop\MongoDB Task\fleet-engine> php cli.php dispatch DRV_002 TRK_003 SHP_004 WH_005
ERROR: Dispatch Blocked. Truck TRK_003 is In-Transit.
PS C:\Users\K MANWITH REDDY\OneDrive\Documents\Desktop\MongoDB Task\fleet-engine>
```

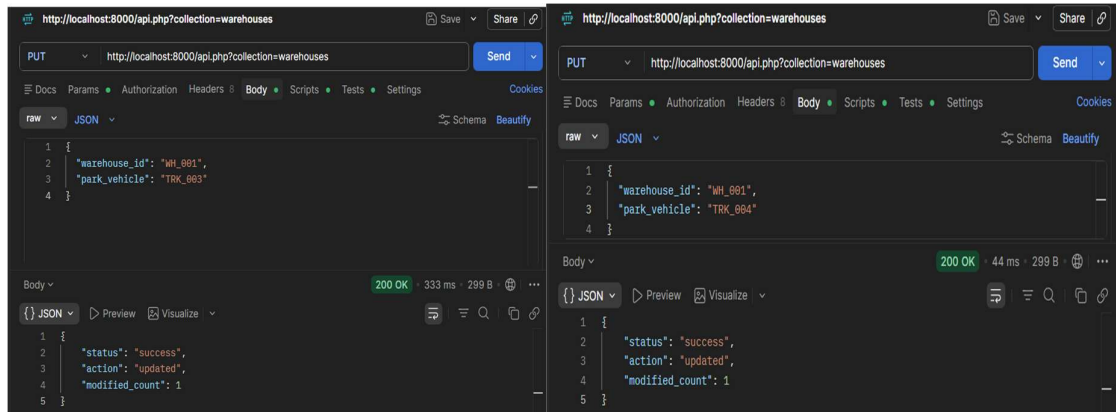
The first snapshot shows the successful validation and state-locking of the truck and shipment. The second snapshot highlights the system's idempotency lock: utilizing a compound B-Tree index, the backend detects the identical repetitive command within a 5-minute threshold and safely aborts the operation, preventing corrupted or duplicated dispatch logs.

Test 4: Defensive Soft-Deletes & Filtering

4. Immutable History via Soft-Delete Architecture

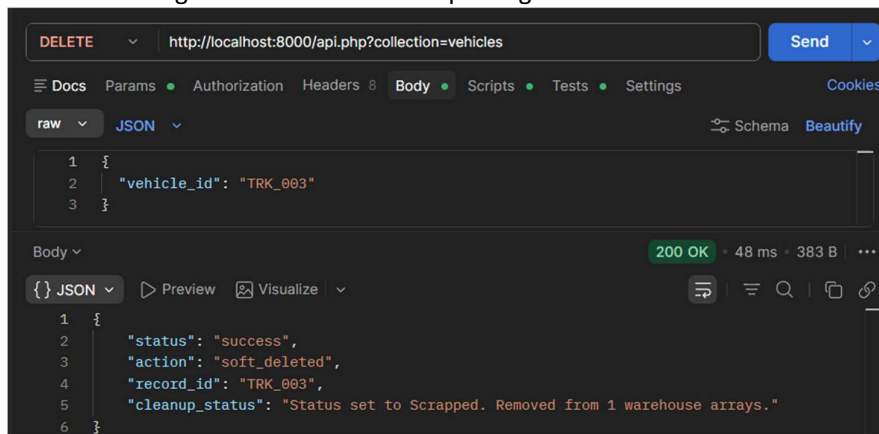
The Action: We will securely delete one of the seeded trucks.

Step 1: Park the Trucks(TRK_003,TRK_004) into the Warehouse.



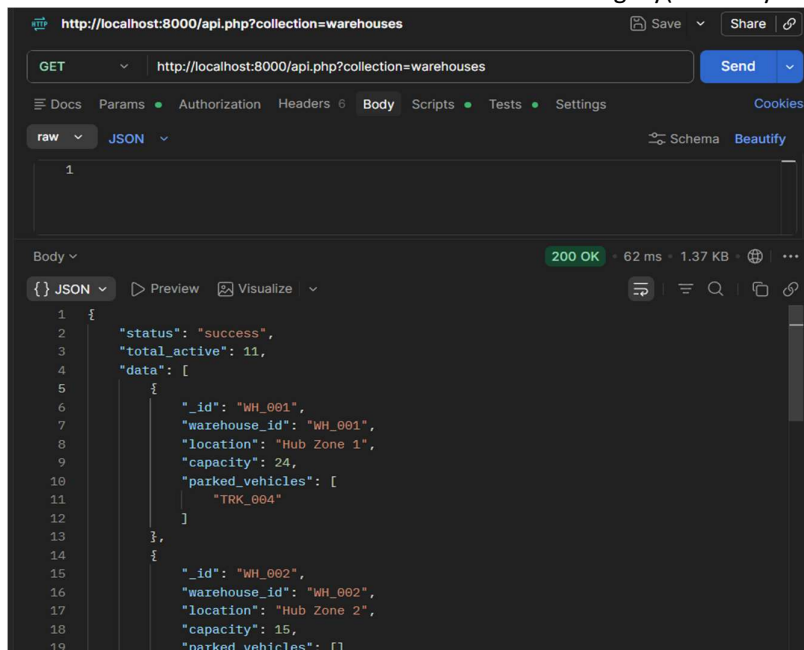
STEP 2: Soft-Delete (Scrap) the Parked Truck

Now, we will simulate a real-world scenario where a truck is suddenly decommissioned or scrapped while it is sitting inside that warehouse parking lot.



STEP-3: Verify the Automated Array Eviction

Automated Cascades: Multi-Collection Referential Integrity(see array of parked_vehicles)



This multi-step execution demonstrates the backend's ability to maintain referential data integrity across disconnected collections without relying on a relational SQL foreign-key engine.

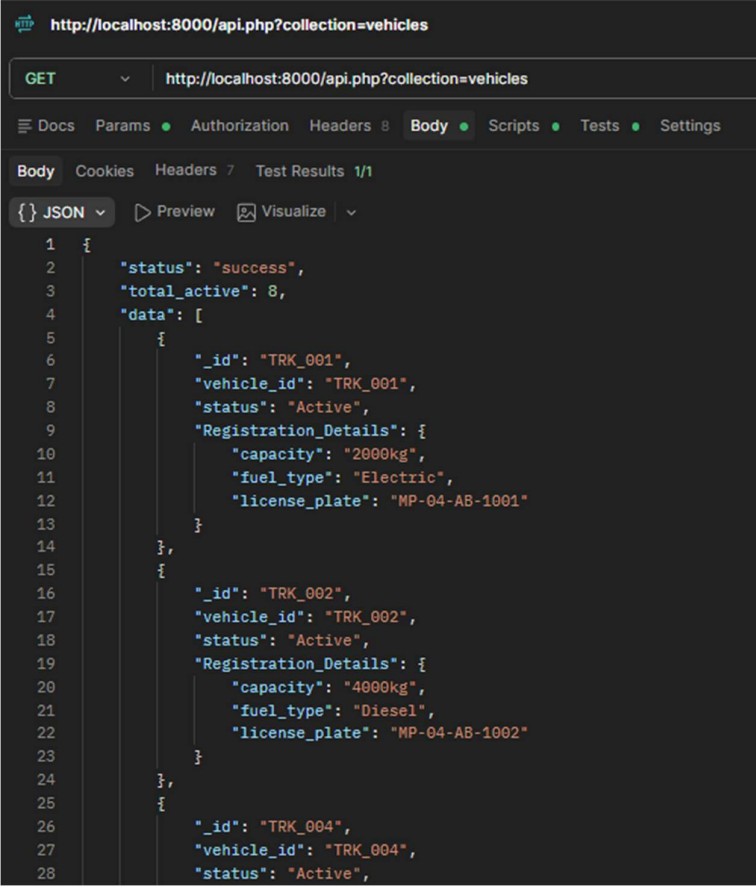
In Step 1, a vehicle (TRK_003) is parked inside a warehouse array via a PUT request.

In Step 2, a DELETE request soft-deletes the vehicle by switching its state to 'Scrapped'. The backend immediately executes a cascading update across the warehouses collection, utilizing the \$pull operator to autonomously eject the vehicle's ID from the warehouse inventory.

Step 3 confirms that the warehouse array has shrunk back to its original size, preventing ghost data or inventory leakage.

Test 5: Global Data Filtering & Dynamic Aggregation(In vechiles collection)

The Action: Since you just scrapped TRK_003, it should no longer appear in the active fleet list.

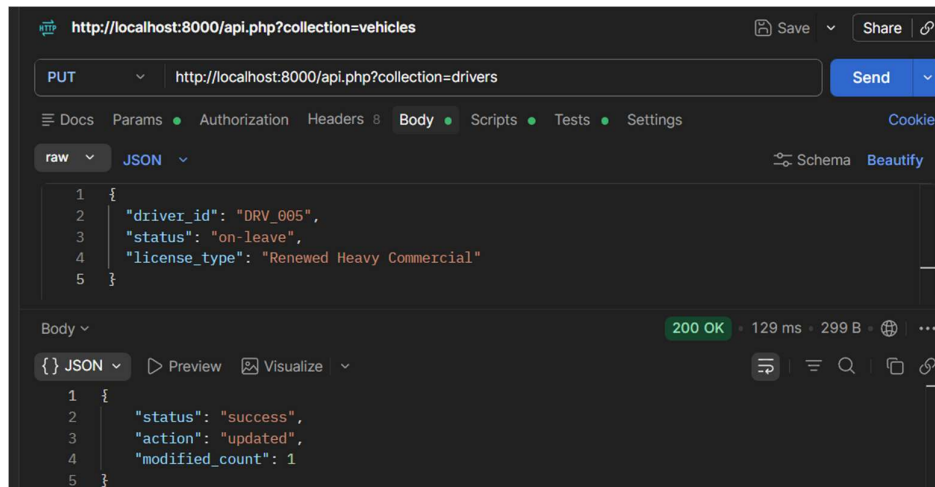


```
http://localhost:8000/api.php?collection=vehicles
GET http://localhost:8000/api.php?collection=vehicles
Body
JSON
1 {
2   "status": "success",
3   "total_active": 8,
4   "data": [
5     {
6       "_id": "TRK_001",
7       "vehicle_id": "TRK_001",
8       "status": "Active",
9       "Registration_Details": {
10        "capacity": "2000kg",
11        "fuel_type": "Electric",
12        "license_plate": "MP-04-AB-1001"
13      }
14    },
15    {
16      "_id": "TRK_002",
17      "vehicle_id": "TRK_002",
18      "status": "Active",
19      "Registration_Details": {
20        "capacity": "4000kg",
21        "fuel_type": "Diesel",
22        "license_plate": "MP-04-AB-1002"
23      }
24    },
25    {
26      "_id": "TRK_004",
27      "vehicle_id": "TRK_004",
28      "status": "Active",
```

This Test verifies the API's data retrieval and obfuscation layer. When a GET request is executed, the backend does not just dump the raw database contents. It applies a global filter (\$nin: ['Scrapped']) to safely hide soft-deleted records from the client application while preserving them in the database. Additionally, the API dynamically aggregates the total_active count (Here, 8) in real-time, completely eliminating the need for manual counter fields.

Test 6: "Update" (PUT)

The Code Line It Tests: \$updateResult = \$collection->updateOne(['_id' => \$recordId], ['\$set' => \$data])



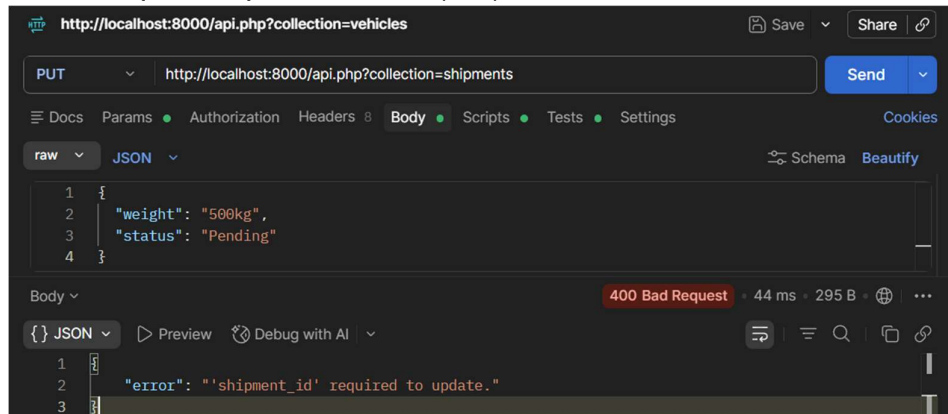
Test 7: The "Bad Request" Guardrails (400 Errors)

we built safeguards to reject requests that forget to include the primary key. We need to prove the API doesn't crash when sent bad data.

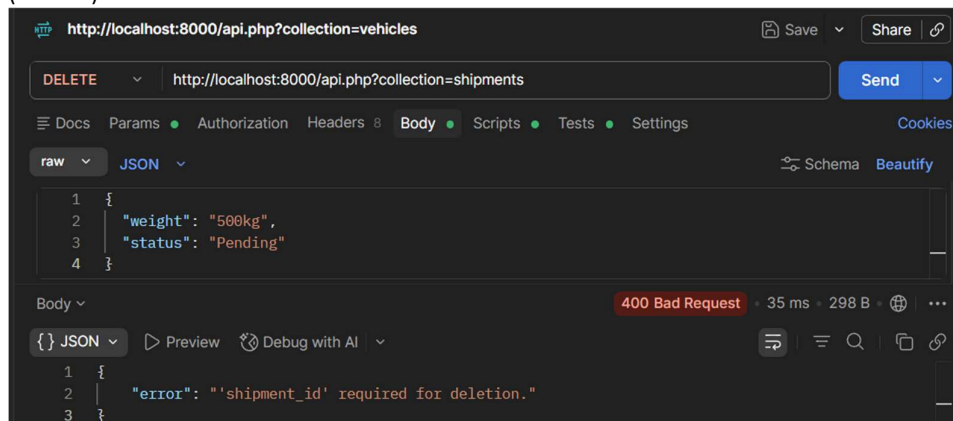
The Code Line It Tests:

```
if (!$data || !isset($data[$idField])) { http_response_code(400); ... }
```

The Action (Postman): Method: POST (PUT)



(DELETE)



Test 8: CLI Syntax Guardrails

The CLI script requires exactly 4 parameters (driver, truck, shipment, warehouse). So need to trigger the block that catches human typos in the terminal.

The Code Line It Tests:

```
if ($argc < 6) { echo "ERROR: Missing parameters..."
```

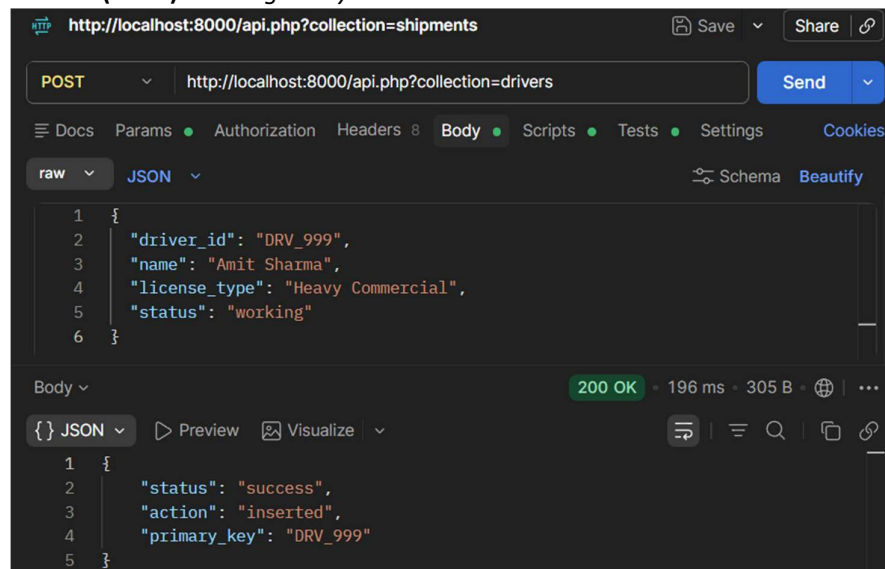
The Action (Terminal): wantedly forget the warehouse ID.

```
PS C:\Users\K MANWITH REDDY\OneDrive\Documents\Desktop\MongoDB Task\fleet-engine> php cli.php dispatch DRV_001 TRK_002 SHP_003
ERROR: Missing parameters.
Usage: php cli.php dispatch <driver_id> <vehicle_id> <shipment_id> <warehouse_id>
PS C:\Users\K MANWITH REDDY\OneDrive\Documents\Desktop\MongoDB Task\fleet-engine>
```

simple CRUD testing...

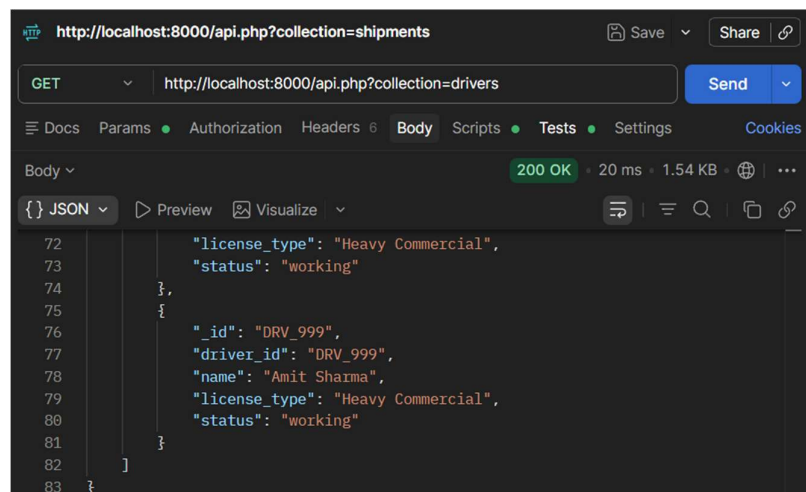
For Driver Collection

CREATE (POST) -Proving the system can insert a new driver with a custom ID.



The API successfully ingests a new driver entity. The custom `driver_id` is natively mapped to MongoDB's immutable `_id` primary key, preventing future duplication.

READ (GET) -.(showing DRV_999 inserted in Database)



UPDATE (PUT) -Proving the system can modify existing text strings safely and its verification.

The first screenshot shows a PUT request to `http://localhost:8000/api.php?collection=drivers` with a JSON body: `{ "driver_id": "DRV_999", "status": "on-leave" }`. The response body shows: `{ "status": "success", "action": "updated", "modified_count": 1 }`.

The second screenshot shows a GET request to `http://localhost:8000/api.php?collection=drivers` with a JSON body: `{ "driver_id": "DRV_999", "status": "on-leave" }`. The response body shows: `{ "_id": "DRV_999", "driver_id": "DRV_999", "name": "Amit Sharma", "license_type": "Heavy Commercial", "status": "on-leave" }`.

DELETE -Proving the soft-delete archive logic works for personnel.

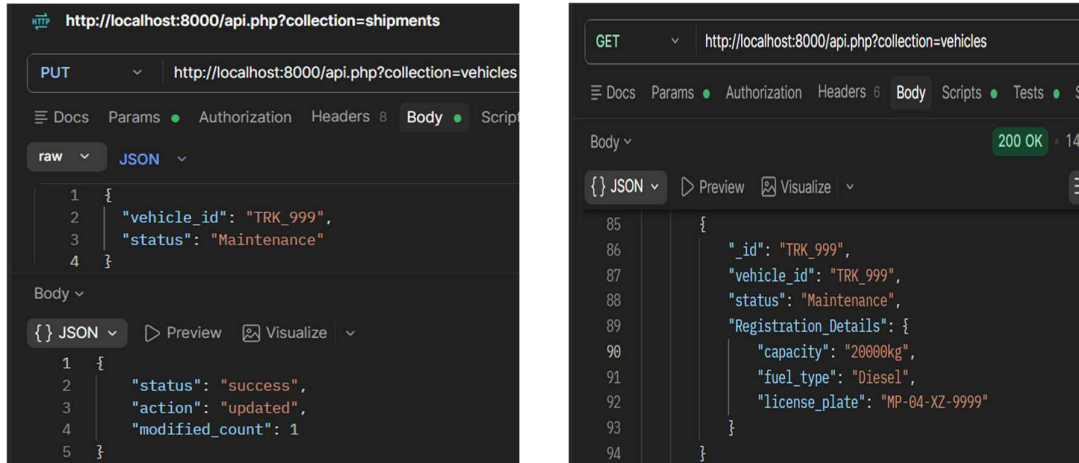
The screenshot shows a DELETE request to `http://localhost:8000/api.php?collection=drivers` with a JSON body: `{ "driver_id": "DRV_999" }`. The response body shows: `{ "status": "success", "action": "soft_deleted", "record_id": "DRV_999", "cleanup_status": "Soft deleted as 'Retired'. No array cleanup needed." }`.

Vehicles Collection

CREATE (POST) -Proving the system can ingest complex, nested JSON schemas.

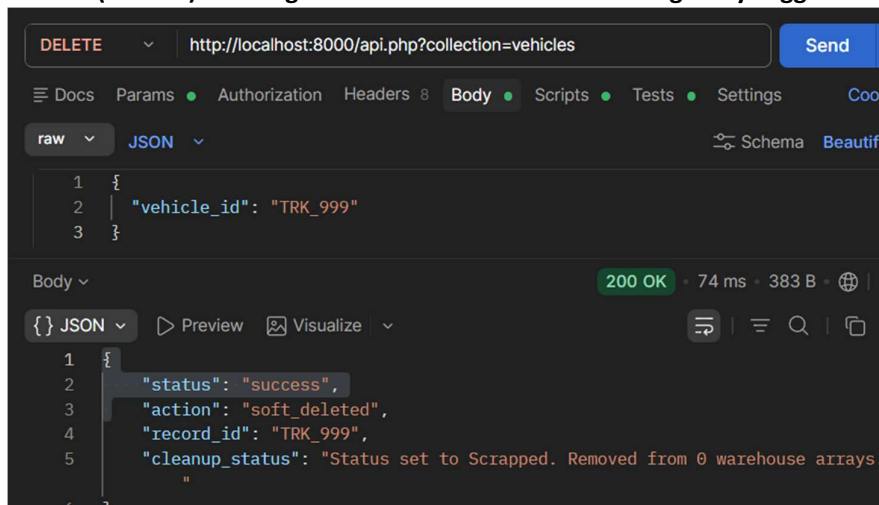
The screenshot shows a GET request to `http://localhost:8000/api.php?collection=vehicles`. The response body shows a complex nested JSON schema: `{ "_id": "TRK_999", "vehicle_id": "TRK_999", "status": "Active", "Registration_Details": { "capacity": "20000kg", "fuel_type": "Diesel", "license_plate": "MP-04-XZ-9999" } }`.

UPDATE (PUT)-Proving state management updates and its verification.



The application accurately updates the operational state of a specific vehicle. By utilizing the precise vehicle_id identifier, the system alters the state to 'Maintenance' while completely protecting the nested registration data from being overwritten.

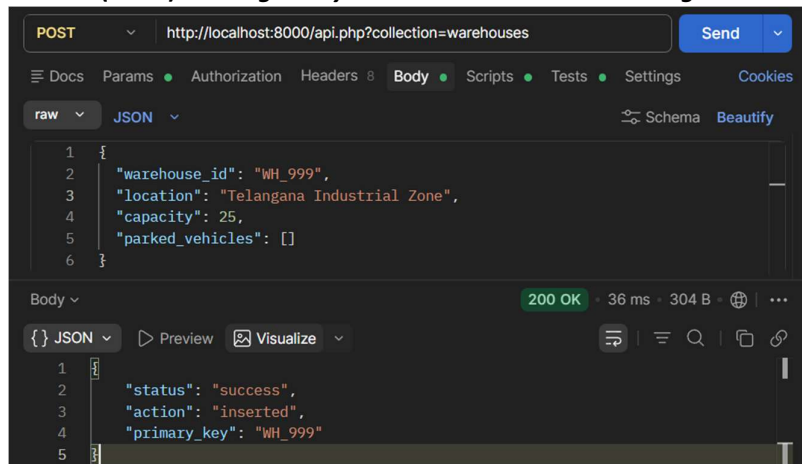
DELETE (DELETE)- Proving the soft-delete and the cascading array trigger.



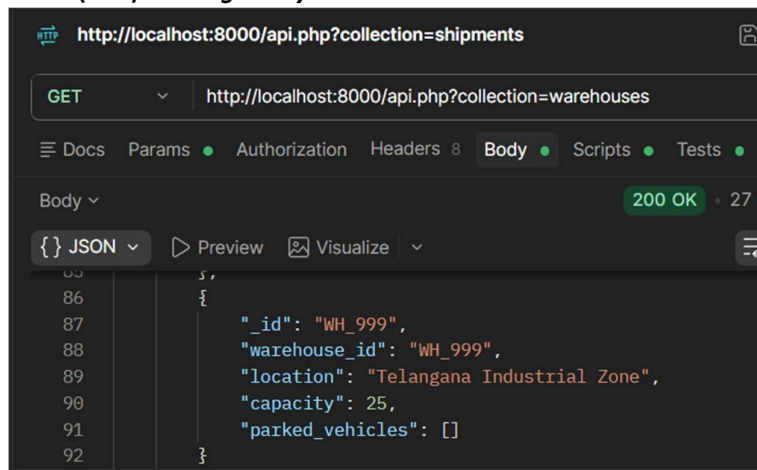
DELETE Operation (Vehicles): The API executes a highly defensive deprecation workflow. The vehicle is soft-deleted by transitioning its status to 'Scrapped'. Crucially, this action automatically fires a multi-collection cascading cleanup mechanism that actively scans all warehouse arrays to ensure the scrapped asset is evicted from any active parking inventories.

Warehouses Collection

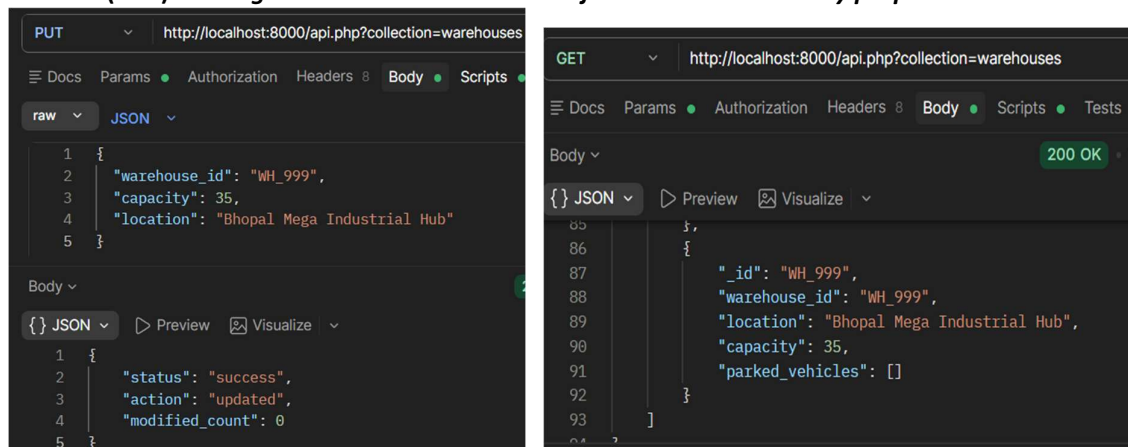
CREATE (POST)-Proving the system can instantiate a new logistics hub location.



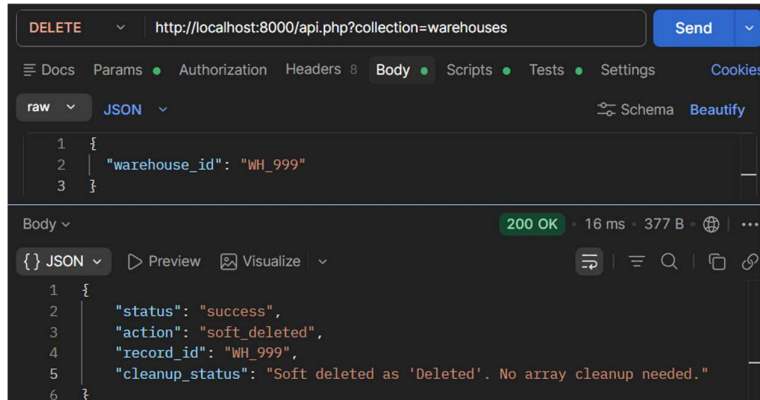
READ (GET)-Proving the system can read and measure the warehouse registry.



UPDATE (PUT)-Proving standard administrative adjustments on non-array properties.

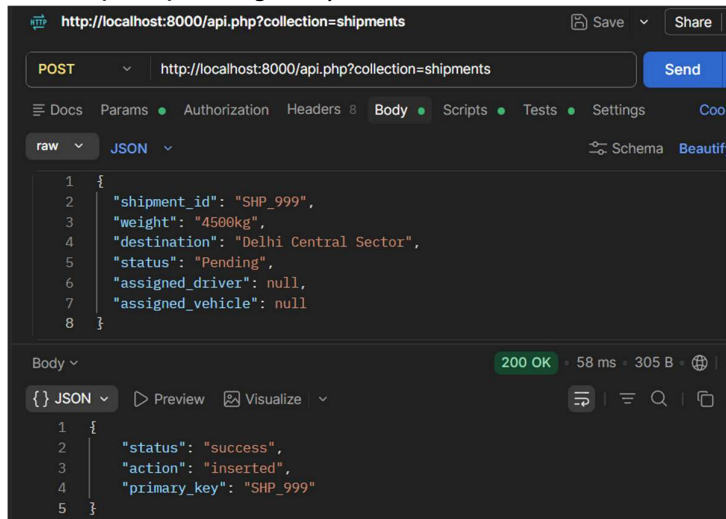


DELETE (DELETE)-Proving isolated soft-deletes on infrastructure.

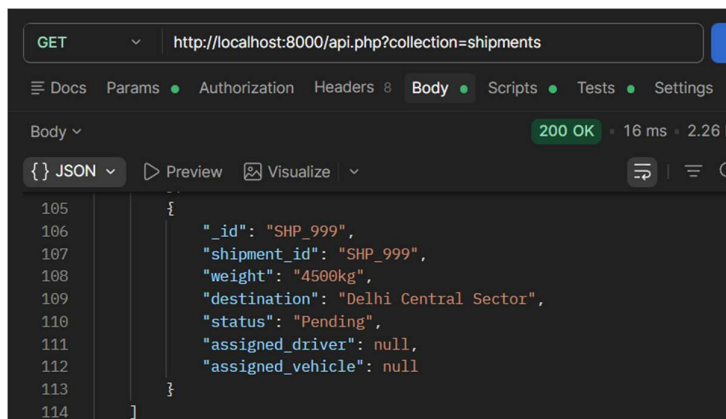


Shipments Collection

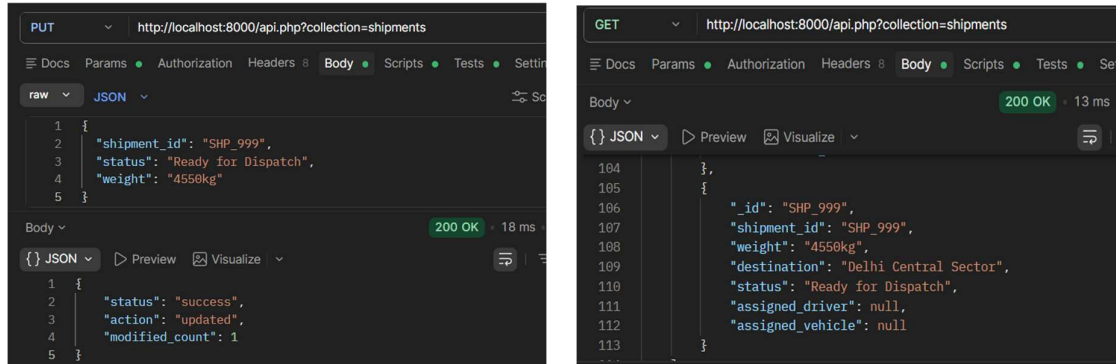
CREATE (POST)-Proving the system can instantiate a new standalone logistics order.



READ (GET)-Proving the system can read and aggregate unassigned orders.

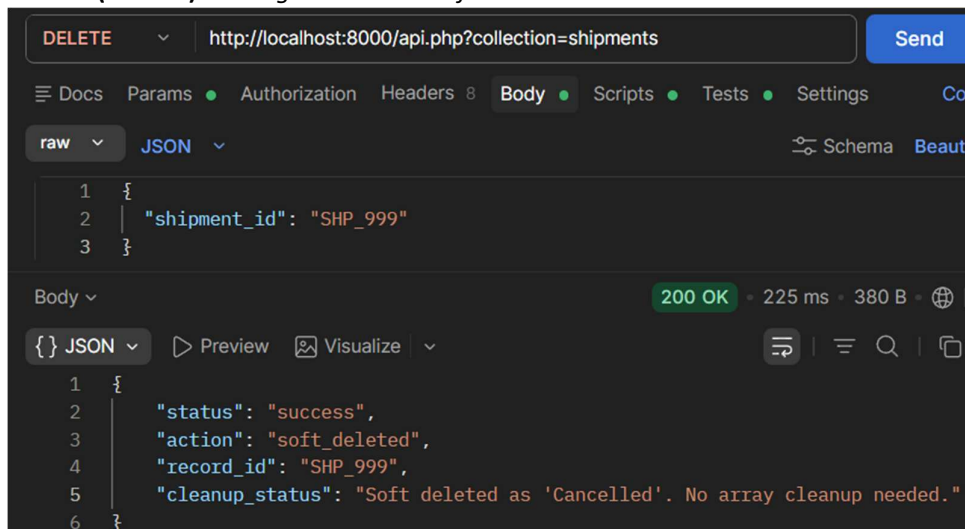


UPDATE (PUT)-In shipments(Order)



This test validates standard metadata modifications. The shipment's status and weight are successfully updated via the \$set operator without corrupting the unassigned driver or vehicle null fields, preserving the integrity of the data model prior to dispatch.

DELETE (DELETE)-Proving the isolated soft-delete architecture.



The API intercepts the deletion command for a logistics order, converting it into a soft-delete by updating the status to 'Cancelled'. Because the shipments collection does not contain complex parent-child inventory arrays, the backend efficiently bypasses the cascading cleanup engine, optimizing execution speed while preserving historical record integrity.